2026/9/5

ACM 格式练习学习笔记

学习内容概览

本次整理 `python/ACM/ACM格式练习` 中的 7 个练习，重点学习 ACM 标准输入输出、动态规划、距离计算、聚类算法、矩阵乘法和 Attention 的基本实现。

文件	主题	核心方法
`动态规划_最小量化误差.py`	最小量化误差	分组代价、动态规划
`KNN邻接.py`	K 近邻分类	欧氏距离、排序、`Counter`
`K-Means.py`	K-Means 聚类	最近中心分配、均值更新
`DBSCAN聚类.py`	DBSCAN 聚类	邻域查询、核心点、BFS
`Attention.py`	简化 Attention	矩阵乘法、转置、行归一化
`A+B问题1.py`	多组 A+B	读到 EOF、异常结束
`A+B问题2.py`	指定组数的 A+B	先读组数、循环读取

动态规划：最小量化误差

问题分析

共有 `N` 层，每层可以选择 `2`、`4` 或 `8` 位量化。每层的量化误差取决于该层权重和选择的位数，同时所有层使用的位数总和不能超过 `Qmax`，目标是让总误差最小。

源码中的 `calc_error(weights, q)` 先计算缩放因子 `2 ** q`，再将权重截断到量化网格，最后累加每个权重的绝对误差。

```
def calc_error(weights, q):
    err = 0
    scale = 2 ** q
    for w in weights:
        wq = int(w * scale)
        wr = wq / scale
        err += abs(w - wr)
    return err
```

状态设计

令 `dp[j]` 表示处理完前若干层、总共使用 `j` 位量化时的最小误差。

处理新的一层时，枚举当前层的量化位数 `q`：

```
new_dp[j + q] = min(new_dp[j + q], dp[j] + error)
```

每次使用新的 `new_dp`，表示当前层只能从上一层的状态转移而来，避免同一层被重复选择。最后在合法的总位数中取最小值。

- 时间复杂度：约为 $O(N \times Qmax \times 3 \times L)$，其中 `L` 是一层中的权重数量。
- 空间复杂度：$O(Qmax)$。
- `INF` 用来表示当前总位数还无法达到。

易错点

- `int()` 是向零截断，不是数学意义上的向下取整；负数尤其需要注意。
- `dp[0] = 0` 是初始状态，其余状态必须初始化为无穷大。
- 最小可用总位数是 `N * 2`，因为每层至少选择 2 位。
- 如果题目要求标准四舍五入，不能直接默认 Python 的 `round` 满足所有场景，应确认题目规则。

KNN 邻接

基本流程

KNN 不需要训练阶段。对测试点和每个样本计算距离，选出距离最小的 `k` 个样本，再使用这些邻居中出现次数最多的标签进行分类。

```
def dist(a, b):
    return sum((a[d] - b[d]) ** 2 for d in range(len(a)))
```

这里使用欧氏距离的平方，因为平方根不会改变距离大小规律，可以减少一次开方运算。

源码先保存所有距离，再排序得到前 `k` 个距离，之后使用 `.index()` 找回样本下标，并通过 `Counter` 统计标签：

```
labels = [sample_label[i] for i in final_index]
counter = Counter(labels)
label, count = counter.most_common(1)[0]
```

- 距离计算复杂度：$O(m \times d)$，`m` 是样本数，`d` 是特征维度。
- 完整排序复杂度：$O(m \log m)$。
- 保存距离和样本信息的空间复杂度：$O(m)$。

易错点

- 样本最后一列是标签，前面的列才是特征，切片时要拆分清楚。

- 不同特征的量纲差异较大时，距离会被大数值特征主导，实际使用前通常要标准化。
- 使用 `dis.index(value)` 遇到相同距离时可能得到同一个下标，严谨做法是对 `(距离, 下标)` 排序：

```
nearest = sorted((distance, i) for i, distance in enumerate(dis))[:k]
final_index = [i for _, i in nearest]
```

- `Counter.most_common(1)[0]` 返回的是 `(标签, 次数)`，不是单独的标签。

K-Means 聚类

算法流程

K-Means 需要指定聚类中心数量 `K`，不断重复以下两个步骤：

1. 将每个点分配给距离最近的中心。
2. 对每个簇中的点求算术平均值，作为新的中心。

```
for p in points:
    min_idx = 0
    min_dist = float('inf')
    for i in range(K):
        d = dist(p, centers[i])
        if d < min_dist:
            min_dist = d
            min_idx = i
    clusters[min_idx].append(p)
```

更新中心时，空簇不能直接求平均，因此源码选择保留原来的中心：

```
if not clusters[i]:
    new_centers.append(centers[i])
```

- 若数据点数量为 `m`、维度为 `d`、迭代次数为 `T`，时间复杂度约为 $O(T \times m \times K \times d)$。
- 保存簇和中心的空间复杂度约为 $O(m + Kd)$。

易错点

- 每次迭代都要重新创建 `clusters`，否则会混入上一次迭代的分组结果。
- 找最近中心时，`min_dist` 必须在处理每个数据点前重新设置为无穷大。
- 空簇需要单独处理。
- 输出 `:.2f` 可以固定保留两位小数；如果题目要求标准四舍五入，需要确认格式化输出是否满足判题规则。
- K-Means 结果依赖初始中心，实际算法通常会多次随机初始化并选择更优结果。

DBSCAN 聚类

核心概念

DBSCAN 通过半径 `eps` 和最小邻居数 `min_samples` 定义聚类：

- 核心点：半径 `eps` 内的点数不少于 `min_samples`。
- 边界点：自身不是核心点，但位于某个核心点的邻域内。
- 噪声点：既不是核心点，也不属于任何核心点扩展出的簇。

源码中的 `region_query(i)` 枚举全部点，找出测试点 `i` 的邻居；找到核心点后，使用队列进行 BFS 扩展：

```
queue = neighbors[:]
while queue:
    j = queue.pop(0)
    if not visited[j]:
        visited[j] = True
        new_neighbors = region_query(j)
        if len(new_neighbors) >= min_samples:
            queue.extend(new_neighbors)
```

如果一个之前被标记为噪声的点后来被某个核心点扩展到，就可以重新归入当前簇，这也是源码中再次判断 `labels[j] == -1` 的原因。

- 每次邻域查询需要 $O(x \times d)$。
- 直接实现中最坏时间复杂度约为 $O(x^2 \times d)$。
- `visited` 和 `labels` 使用 $O(x)$ 空间，队列最多也可能保存 $O(x)$ 个点。

易错点

- `eps` 是距离阈值，代码比较的是距离平方和 `eps ** 2`，两者要保持一致。
- 点自身通常也包含在邻域中，因此 `min_samples` 的含义必须按照题目定义处理。
- 噪声是暂时状态，后续可能被重新归入簇。
- `pop(0)` 操作会移动列表元素，数据量较大时应使用 `collections.deque` 的 `popleft()`。
- `labels = [-1] * x` 同时表示“未分类”和“噪声”，规模更大的实现可以使用单独的未分类标记。

Attention：矩阵乘法实现

矩阵形状

输入为 `n, m, h` 时：

矩阵	形状
`X`	`n × m`
`W1`, `W2`, `W3`	`m × h`
`Q`, `K`, `V`	`n × h`

矩阵	形状
K.T	h × n
S = QK.T	n × n
Y = softmax(S)V	n × h

矩阵乘法的形状规则是：左矩阵列数必须等于右矩阵行数，结果形状为“左矩阵行数 × 右矩阵列数”。

```
def matmul(A, B):
    rows = len(A)
    cols = len(B[0])
    mid = len(B)
    C = [[0.0 for _ in range(cols)] for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            for k in range(mid):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

计算过程

1. 构造输入矩阵 `X` 和三个权重矩阵。
2. 计算 `Q`、`K`、`V`。
3. 计算缩放点积 `QK.T / sqrt(h)`。
4. 对每一行进行归一化。
5. 用归一化后的权重加权 `V`，得到 `Y`。
6. 求 `Y` 的全部元素之和并输出四舍五入结果。

需要注意：源码中的“softmax”实际上是“每行除以行和”，并没有执行 `exp`。这是题目定义的归一化方式，不是严格意义上的 Softmax。标准 Softmax 通常为：

```
exp_score = math.exp(score)
softmax(score_i) = exp(score_i) / sum(exp(score_j))
```

- 三重循环矩阵乘法复杂度：若矩阵规模为 `a × b` 和 `b × c`，则为 $O(abc)$。
- 转置使用 `zip(*A)`，会把原矩阵的列转换成行。
- 矩阵乘法和中间矩阵会占用与矩阵规模相关的空间。

易错点

- `zip(*A)` 的 `*` 是参数解包，用于把每一行作为独立参数传入 `zip`。
- 计算 `QK.T` 时必须先转置 `K`，否则形状不匹配。
- 矩阵乘法中的中间维度必须相等。
- 行归一化前要确认行和不为 0。
- 代码文件中保留了多个版本，实际提交 ACM 题目时应只保留一个完整程序，避免重复读取输入。

ACM 输入输出：A+B 问题

A+B 问题 1：读到 EOF

每行包含两个整数，不预先给出数据组数。程序不断读取输入，遇到文件结束时停止：

```
while True:
    try:
        a, b = map(int, input().split())
        print(a + b)
    except:
        break
```

这种写法可以处理在线评测中的 EOF，但裸 `except` 会吞掉所有异常。更清晰的写法是捕获 `EOFError`，并单独处理格式错误。

A+B 问题 2：先读取组数

第一行给出 `n`，之后读取 `n` 组整数：

```
try:
    n = int(input())
    for _ in range(n):
        a, b = map(int, input().split())
        print(a + b)
except EOFError:
    pass
```

`map` 知识点

- `map(function, iterable)` 会对可迭代对象中的每个元素执行函数。
- `map` 返回迭代器，不是列表；需要列表时使用 `list(map(...))`。
- `map(int, input().split())` 是 ACM 中常见的"读取一行整数"写法。
- `lambda` 可以快速构造简单函数，例如 `map(lambda x: x ** 2, nums)`。

本次学习中的通用方法总结

1. 先确认输入结构

ACM 题目中，输入可能是固定组数、读到 EOF，或者包含多行矩阵。写代码前应先确认每一行的含义、数量和数据类型。

2. 用状态表示"处理到哪里"

动态规划中的 `dp[j]` 表示已使用的资源量和当前最优值。设计状态时要明确：状态含义、初始值、转移方式和最终答案范围。

3. 先抽象距离函数

KNN、K-Means 和 DBSCAN 都依赖距离计算。将距离单独封装后，主流程更容易阅读，也方便从二维扩展到任意维度。

4. 检查数据结构形状

处理矩阵时，要在每次乘法前写出矩阵形状。形状检查能够快速发现转置、行列数和嵌套列表层级方面的问题。

5. 关注边界状态

- 动态规划的不可达状态。
- K-Means 的空簇。
- DBSCAN 的噪声点重新归类。
- KNN 的相同距离和投票平局。
- Attention 的零行和。
- ACM 输入中的 EOF。

Python 工具速查

写法	作用
`map(int, input().split())`	读取一行整数
`list(map(float, values))`	将多个字符串转为浮点数列表
`sorted(items)`	返回排序后的新列表
`Counter(values)`	统计元素频次
`float('inf')`	表示正无穷，常用于最小值初始化
`zip(*matrix)`	转置二维列表
`f'{x:.2f}'`	格式化为两位小数
`sum(expression for item in items)`	对生成器表达式求和